

RELATÓRIO TÉCNICO — TECH CHALLENGE FASE 2

1. Introdução

Este relatório apresenta o desenvolvimento de uma API REST para gestão de biblioteca, com foco no controle de livros, usuários, empréstimos e relatórios operacionais. A solução foi implementada em Java 21 com Spring Boot, adotando arquitetura em camadas, persistência relacional com PostgreSQL, documentação com OpenAPI e execução containerizada com Docker.

O objetivo do trabalho foi construir uma aplicação consistente do ponto de vista funcional e técnico, capaz de atender aos requisitos propostos no Tech Challenge com organização estrutural, regras de negócio bem definidas e qualidade de código compatível com um cenário profissional de desenvolvimento.

2. Problema e Objetivo da Solução

A administração manual ou fragmentada de bibliotecas tende a gerar problemas recorrentes, como perda de controle sobre a disponibilidade dos livros, inconsistências no registro de empréstimos e devoluções, dificuldade de rastreamento das operações e baixa visibilidade sobre o uso do acervo.

Diante desse contexto, a solução proposta teve como objetivo centralizar essas operações em uma API REST capaz de registrar dados de forma padronizada, aplicar regras de negócio de maneira consistente e disponibilizar informações consolidadas para apoio à operação da biblioteca.

3. Escopo Funcional Implementado

O sistema desenvolvido contempla as seguintes funcionalidades:

- cadastro, consulta, atualização e exclusão de livros;
- consulta paginada de livros com filtros opcionais por título, autor e ISBN;
- cadastro, consulta, atualização e exclusão de usuários;
- registro de empréstimos com cálculo automático da data prevista de devolução;
- registro de devoluções com atualização do status do empréstimo e da disponibilidade do livro;
- consulta paginada de empréstimos por status;
- geração de relatório com os 20 livros mais emprestados;
- geração de relatório com os livros emprestados no momento e sua respectiva previsão de devolução.

Funcionalidades como autenticação, autorização, interface web e deploy em nuvem não fazem parte do escopo implementado nesta fase e foram tratadas apenas como possibilidades de evolução futura.

4. Arquitetura e Organização do Projeto

A aplicação foi estruturada em camadas, com separação clara de responsabilidades entre os componentes:

- Controller: expõe os endpoints HTTP e delega o processamento para a camada de serviço;
- Service: concentra as regras de negócio, validações operacionais e controle transacional;
- Repository: realiza o acesso a dados com Spring Data JPA e consultas específicas quando necessário;
- DTO: define os contratos de entrada e saída da API, evitando exposição direta das entidades;
- Mapper: executa a conversão explícita entre entidades e DTOs;
- Exception: padroniza exceções de domínio e o tratamento global de erros.

Essa organização reduz acoplamento entre camadas, facilita manutenção e torna o fluxo da aplicação mais previsível. Os controllers permanecem enxutos, enquanto as regras críticas ficam centralizadas nos serviços.

5. Modelagem de Domínio

O domínio foi modelado a partir de três entidades principais: Livro, Usuario e Emprestimo.

A entidade Livro armazena identificador, título, autor, ISBN e indicador de disponibilidade. O ISBN foi definido como único para evitar duplicidade de cadastro. A entidade Usuario contém identificador, nome e e-mail, também com restrição de unicidade para o e-mail.

A entidade Emprestimo representa a operação de empréstimo propriamente dita e relaciona um livro a um usuário. Além dos vínculos, ela registra data de empréstimo, data prevista de devolução, data efetiva de devolução e status. O status é representado pelo enum StatusEmprestimo, com os valores EMPRESTADO e DEVOLVIDO.

Os relacionamentos foram modelados de forma a permitir histórico: um livro pode possuir múltiplos empréstimos ao longo do tempo, assim como um usuário pode possuir múltiplos empréstimos. As entidades foram mantidas sem lógica de negócio, o que preserva a clareza do domínio e evita responsabilidades indevidas na camada de persistência.

6. Regras de Negócio Implementadas

As principais regras de negócio do sistema foram implementadas na camada de serviço.

Para a operação de empréstimo, o sistema valida a existência do livro e do usuário, verifica se o livro está disponível e, em caso positivo, cria um novo empréstimo com data de empréstimo igual à data atual, data prevista de devolução calculada com base na quantidade de dias informada e status inicial EMPRESTADO. Na mesma transação, o livro passa a ser marcado como indisponível.

Para a devolução, o sistema recupera o empréstimo solicitado, valida se ele ainda está com status EMPRESTADO e registra a devolução com a data atual. Em seguida, o status do empréstimo é atualizado para DEVOLVIDO e o livro volta a ficar disponível.

Também foram implementadas validações de unicidade para ISBN e e-mail, com normalização dos valores de entrada para reduzir inconsistências causadas por diferenças de caixa ou espaços em branco. Os DTOs de entrada aplicam ainda restrições de tamanho e limites de negócio, como validação máxima para duração do empréstimo e tamanho dos campos textuais. Além disso, a

exclusão de livros e usuários com empréstimos vinculados é bloqueada para preservar integridade histórica dos dados.

Outro cuidado adotado foi a proteção explícita do estado de disponibilidade do livro: esse atributo não é aceito nos contratos de escrita da API e só pode ser alterado pelas operações transacionais de empréstimo e devolução. Somado a isso, a aplicação utiliza bloqueio pessimista nas operações sensíveis de empréstimo e devolução, evitando cenários em que requisições simultâneas possam gerar estados inconsistentes.

7. Persistência e Acesso a Dados

A persistência foi implementada com Spring Data JPA sobre PostgreSQL. Os repositories foram mantidos objetivos, sem regras de negócio, e concentram apenas as operações necessárias para o funcionamento da aplicação.

No repositório de livros, foi implementada uma consulta paginada com filtros opcionais por título, autor e ISBN. No repositório de usuários, foram mantidas operações específicas para verificação de duplicidade e busca por e-mail. No repositório de empréstimos, foram implementadas consultas por status, consultas de apoio a relatórios e verificações de existência de vínculos para impedir exclusões inconsistentes.

As consultas de relatório foram escritas de forma explícita, com uso de JPQL e carregamento adequado das associações quando necessário. Esse cuidado reduz o risco de consultas adicionais desnecessárias e melhora a previsibilidade do acesso a dados.

8. Contratos da API e Tratamento de Erros

Os contratos de entrada e saída da API foram definidos por DTOs dedicados aos recursos e fluxos principais, desacoplando a camada HTTP das entidades persistidas. No caso de livros, o contrato de escrita não expõe o campo de disponibilidade, preservando a invariante de domínio de que esse estado é controlado apenas pelos fluxos de empréstimo e devolução. As entradas também utilizam Jakarta Validation para verificar formato, obrigatoriedade, tamanho e limites de negócio.

As falhas da aplicação são tratadas de forma padronizada por um handler global de exceções, com registro em log para apoio à rastreabilidade operacional. O sistema diferencia recursos não encontrados, violações de regra de negócio, erros de validação, parâmetros inválidos, requisições malformadas e violações de integridade de dados. As respostas retornam estrutura uniforme com timestamp, status HTTP, mensagem e caminho da requisição, o que favorece manutenção e consumo da API.

9. Documentação da API

Como resultado, a aplicação disponibiliza documentação navegável por meio do Swagger UI nos ambientes de desenvolvimento e demonstração, o que facilita a inspeção dos contratos e a execução de testes manuais. No perfil de produção, a documentação interativa é desabilitada como medida de endurecimento operacional.

10. Containerização e Execução

A configuração central do projeto está no arquivo `application.yml`, que define parâmetros padrão para ambiente local, um perfil específico para execução com Docker e um perfil de produção com

configuração mais restritiva. Também foram explicitados limites padrão de paginação para garantir previsibilidade de consumo da API.

O Dockerfile utiliza build em múltiplas etapas, separando a compilação da aplicação da imagem final de execução. Já o docker-compose.yml define dois serviços: a API e o banco PostgreSQL. Essa configuração permite subir o ambiente completo com um único comando, simplificando a execução e reduzindo diferenças entre ambientes de desenvolvimento e demonstração. No perfil de produção, o esquema do banco é validado na inicialização, em vez de atualizado automaticamente.

11. Estratégia de Testes e Cobertura

A estratégia de testes priorizou as áreas com maior valor funcional e maior risco de regressão: regras de negócio, contratos HTTP e tratamento de exceções.

Na camada de serviços, foram criados testes cobrindo fluxos positivos e cenários de falha para livros, usuários, empréstimos e relatórios. Esses testes verificam validações, unicidade, consistência entre disponibilidade do livro e status do empréstimo, além das regras de exclusão, devolução e atualização segura de livros já emprestados.

Na camada web, foram implementados testes com MockMvc para validação dos endpoints principais, códigos HTTP, serialização das respostas e tratamento de entrada inválida. O tratamento global de exceções também foi exercitado para confirmar o formato padronizado das respostas de erro.

Além dos testes unitários e da camada web, foram adicionados testes de integração com @SpringBootTest e banco H2 em memória configurado em modo compatível com PostgreSQL. Esses testes validam a persistência do fluxo completo de empréstimo e devolução, garantem que a atualização de um livro não reabra sua disponibilidade de forma indevida e exercitam um cenário concorrente em que apenas uma requisição consegue emprestar o mesmo exemplar.

A cobertura foi medida com JaCoCo. No estado final do projeto, a cobertura monitorada ficou em 95,17% no escopo definido pela configuração do build, acima da meta mínima estabelecida. A suíte automatizada totaliza 58 testes aprovados. A regra de cobertura exclui DTOs, entidades, enums, classes de configuração e a classe principal, concentrando a métrica nas camadas com lógica de negócio e orquestração.

12. Considerações Técnicas Finais

Do ponto de vista técnico, a solução resultante apresenta coerência entre modelagem, arquitetura e regras de negócio. O código foi mantido simples, com separação adequada de responsabilidades, mapeamentos explícitos e uso moderado de abstrações. Não foram introduzidos frameworks ou padrões desnecessários ao escopo do projeto.

O resultado final atende ao objetivo de entregar uma base sólida, legível e pronta para evolução incremental. A API está documentada, validada por testes automatizados em múltiplas camadas, preparada para execução com banco real em runtime e com banco em memória nos testes de integração, além de contar com controles de concorrência e configuração de produção mais restritiva.

13. Conclusão

O projeto atendeu ao escopo definido para o Tech Challenge ao entregar uma API REST para gestão de biblioteca com operações essenciais de cadastro, consulta, empréstimo, devolução e relatórios. A implementação buscou conciliar clareza arquitetural, consistência de regras de negócio e qualidade técnica do código.

Mais do que cumprir os requisitos funcionais, o trabalho evoluiu para uma solução com documentação, padronização de erros, ambiente containerizado e testes automatizados com alta cobertura. Esses elementos reforçam a maturidade da entrega e demonstram preocupação com manutenção, confiabilidade e capacidade de evolução do sistema.

14. Evoluções Possíveis

Como continuidade natural do projeto, destacam-se as seguintes possibilidades de evolução:

- autenticação e autorização por perfis de acesso;
- uso de ferramentas de migração de banco, como Flyway ou Liquibase;
- adoção de projections tipadas para relatórios agregados;
- pipeline de integração contínua com execução automática de testes e cobertura;
- deploy em ambiente de nuvem e integração com interface frontend.